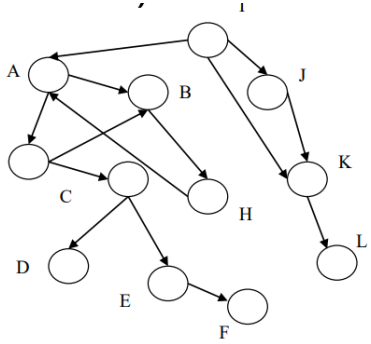
Cheatsheet PA

GRAFURI

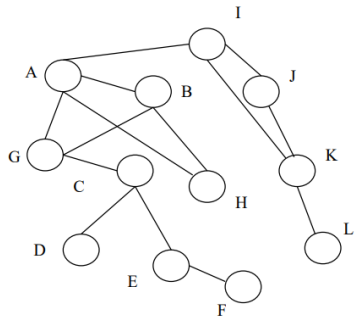
Tipuri - orientate + neorientate

Orientate



Noduri: A, B, C ... L
Arce: AB, AG, CD ...

Neorientate



Noduri: A, B, C ... L
Muchii: AB, AI, BG ...

Notații

G = (V,E);

V – mulțimea de noduri;

E – mulțimea de muchii / arce;

(u,v) – arcul / muchia u,v;

u..v – drum de la u la v; dacă există mai multe variante notăm u..x..v, u..y..v;

R(u) - reachable(u) = mulțimea nodurilor ce pot fi atinse pe căi ce pleacă din u;

- succs(u) – mulțimea succesorilor lui u (graf orientat) sau mulțimea nodurilor adiacente lui u (graf neorientat);
- c(u) – culoarea nodului – specifică starea nodului la un anumit moment al parcurgerii:
  - Alb – nedescoperit;
  - Gri – descoperit, în curs de prelucrare;
  - Negru – descoperit și terminat (cu semnificații diferite pentru BFS si DFS).
- p(u) (π(u)) – "părintele lui u" – identificator al nodului din care s-a ajuns în nodul u prima oară.

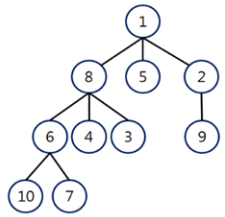
Parcurgere BFS -> COADĂ

Determină numărul minim de muchii / arce între un nod s (nod de start / sursă) și orice nod din V. Acesta reprezintă cel mai scurt drum dacă graful nu are asociat o funcție de costuri.

Pseudocod

BFS(s,G)
- **Pentru fiecare** nod u (u ∈ V)
  - p(u) = null; dist(s,u) = inf; c(u) = alb; // inițializări
- Q = (); // se folosește o coadă în care reținem nodurile de prelucrat
- dist(s,s) = 0; // actualizări: distanța de la sursă până la sursă este 0
- Q ← Q + s; // adăugăm sursa în coadă → începem prelucrarea lui s
- c(s) = gri; // și atunci culoarea lui devine gri
- **Cât timp** (!empty(Q)) // cât timp mai am noduri de prelucrat
  - u = top(Q); // se determină nodul din vârful cozii
  - **Pentru fiecare** nod v (v ∈ succs(u)) // pentru toți vecinii
    - **Dacă** c(v) este alb // nodul nu a mai fost găsit, nu e în coadă
      - **Atunci** { dist(s,v) = dist(s,u) + 1; p(v) = u; c(v) = gri; Q ← Q + v;} // actualizăm structura date
  - c(u) = negru; // am terminat de prelucrat nodul curent
  - Q ← Q - u; // nodul este eliminat din coadă

Practic ia / extrage câte un nod de la începutul cozii și adaugă la finalul acesteia toți vecinii lui nevizitați. Se repetă procedeul până când coada este vidă.



BFS: 1 8 5 2 6 4 3 9 10 7

COMPLEXITATE : O(n+m)
n = număr noduri
m = număr muchii

Parcurgere DFS ->

STIVĂ / RECURSIVITATE (simulează stiva)

Cheatsheet PA

d(u) = momentul descoperirii nodului (se trece prima oară prin u și e totodată și momentul începerii explorării zonei din graf ce poate fi atinsă din u).

f(u) = timpul de finalizare al nodului (momentul în care prelucrarea nodului u a luat sfârșit)

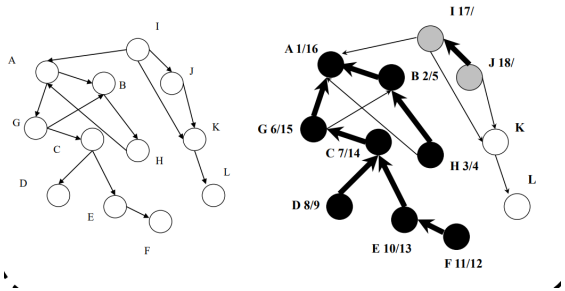
- Tot subarborele de adâncime dominat de u a fost explorat.
- Alternativ: tot subgraful accesibil din u a fost descoperit și finalizat deja.

Pseudocod

DFS(G)
- V = noduri(G)
- **Pentru fiecare** nod u (u ∈ V)
  - c(u) = alb; p(u) = null; // inițializare structură date
- timp = 0; // reține distanța de la rădăcina arborelui DFS până la nodul curent
- **Pentru fiecare** nod u (u ∈ V)
  - **Dacă** c(u) este alb
    - **Atunci** explorare(u); // explorez nodul

explorare(u)
- d(u) = ++ timp; // timpul de descoperire al nodului u
- c(u) = gri; // nod în curs de explorare
- **Pentru fiecare** nod v (v ∈ succs(u)) // încerc sa prelucrez vecinii
  - **Dacă** c(v) este alb
    - **Atunci** {p(v) = u; explorare(v);} // dacă nu au fost prelucrați deja
- c(u) = negru; // am terminat de explorat nodul u
- f(u) = ++ timp; // timpul de finalizare al nodului u

A 1/16 => d(A) = 1 (timpul descoperirii)
f(A) = 16 (timpul finalizării)



I(u) = intervalul de prelucrare al nodului (d(u),f(u)).

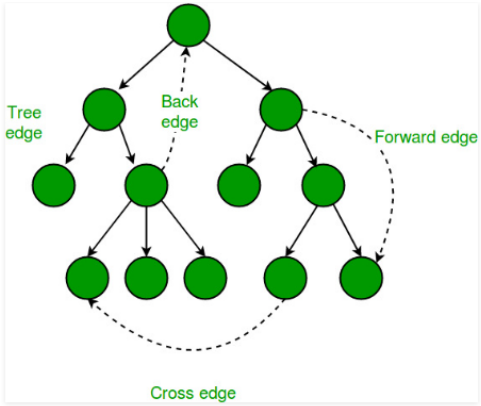
Teorema 5.2. G = (V,E); DFS(G) sparge graful G într-o pădure de arbori Arb(G) = { Arb(u); p(u) = null } unde Arb(u) = (V(u),E(u));
- V(u) = { v | d(u) < d(v) < f(u) } + {u};
- E(u) = { (v, z) | v, z ∈ V(u) && p(z) = v }.

Teorema 5.3. Dacă DFS(G) generează 1 singur arbore => G este conex. (Reciproca este adevărată?)
- Graf orientat conex înseamnă că prin transformarea arcelor în muchii se obține un graf neorientat conex.

Teorema 5.5. ∀ u, v ∈ V, atunci v ∈ V(u) ⇔ I(v) ⊂ I(u).

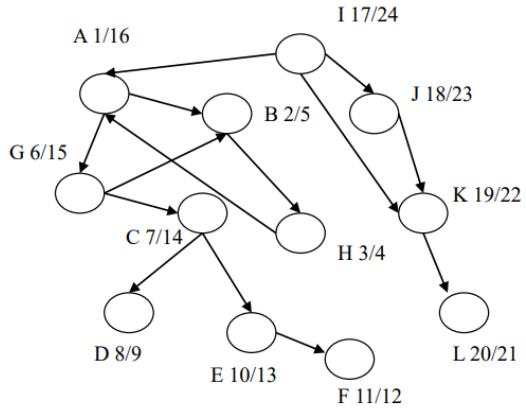
Clasificări arce de grafuri

- **Arc direct (de arbore) = Tree edge** (u, v) - între nod gri și nod alb;
- **Arc invers (de ciclu) (u, v) = Back edge** - între nod gri și nod gri;
- **Arc înainte (u, v) = Forward edge** - nod gri și nod negru și d(u) < d(v);
- **Arc transversal (u, v) = Cross edge** - nod gri și nod negru și d(u) > d(v).



Exemplu:

Arc direct (de arbore):
AB, BH, AG, GC, CD, CE, EF, IJ, JK, KL
Arc înainte:
IK
Arc invers (de ciclu):
HA
Arc transversal:
GB, IA



Teorema 5.6. Teorema drumurilor albe:
- G = (V,E); Arb(u); v este descendent al lui u în Arb(u) ⇔ la momentul d(u) există o cale numai cu noduri albe u..v.

Cheatsheet PA

Teorema 5.7. Într-un graf neorientat, DFS poate descoperi doar muchii directe și inverse.

- Dem prin considerarea cazurilor posibile!
- Fie muchia (u,v) ∈ E și pp. d(u) < d(v). Muchia poate fi străbătută din u sau din v:
  - Caz (u,v): c(u) = gri, c(v) = alb → muchie directă
  - Caz (v,u): la d(u) ∃ o cale cu noduri albe u..v → (Teorema drumurilor albe) v este descendent al lui u în Arb(u) → (Teorema 5.5) d(u) < d(v) < f(v) < f(u) → în intervalul (d(v), f(v)) când se investighează (v,u) c(u) = c(v) = gri → muchie inversă

Teorema 5.8. G = graf orientat; G ciclic ⇔ în timpul execuției DFS găsim arce inverse.

COMPLEXITATE: O(n+m)

n = număr de noduri

m = număr de muchii

Sortare topologică

- G = (V,E) orientat, aciclic.
- V_S – secvența de noduri a.î. ∀(u,v) ∈ E, avem index(u) < index(v).
- Scop: Sortare_topologică(G) => V_S.

Idee bazată pe DFS:

- G = (V,E) orientat, aciclic; la sfârșitul DFS avem ∀(u,v) ∈ E, f(v) < f(u)
- => colectăm în V_S vârfurile în ordinea descrescătoare a timpilor f

Pseudocod

Sortare_topologică (G)

- **Pentru fiecare** nod u (u ∈ V) {c(u) = alb;} // inițializări
- V_S = ∅;
- **Pentru fiecare** nod u (u ∈ V) // pentru fiecare componentă conexă
  - **Dacă** c(u) este alb
    - V_S = Explorează (u, V_S) // prelucrez componenta conexă
- **Întoarce** V_S

Explorează (u, V_S)

- c(u) = gri // prelucrez nodul, deci îi actualizez culoarea
- **Pentru fiecare** nod v (v ∈ succs(u))
  - **Dacă** c(v) este alb **atunci** V_S = Explorează (v, V_S) // recursivitate
  - **Dacă** c(v) este gri **atunci Întoarce** Eroare: graf ciclic
- c(u) = negru // am terminat prelucrarea nodului
- **Întoarce** cons(u, V_S) // inserează nodul u la începutul lui V_S

COMPLEXITATE: O(n+m)

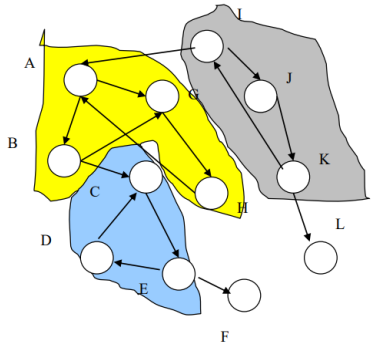
n = număr de noduri

m = număr de muchii

COMPONENTE TARE CONEXE (CTC)

Definiție: Fie G = (V,E) un graf orientat. G este tare-conex ⇔ ∀ u,v∈V ∃ o cale u..v și o cale v..u (u∈R(v) și v∈R(u)).

Definiție: G = (V,E) graf orientat. G' = (V',E'), V'⊆V, E'⊆E. G' este o CTC a lui G ⇔ G' e tare-conex (∀ u,v∈V', u∈R(v) și v∈R(u)) și G' este maximal (ca număr de noduri).



Strămoș

Definiție: G = (V,E) orientat, u∈V. Φ(u) = strămoș DFS al lui u determinat în cursul DFS(G) dacă:

- Φ(u)∈R(u)
- f(Φ(u)) = max{f(v) | v∈R(u)}

Practic este primul nod din CTC descoperit în DFS.

Teorema 5.13. G = (V,E) orientat, ∀ u,v∈V; u și v aparțin aceleiași CTC ⇔ Φ(u) = Φ(v).

Primul nod dintr-o CTC descoperit prin DFS va avea drept succesori în arborele generat de DFS toate elementele componentei conexe!

Algoritmul lui Kosaraju

CTC(G)

- DFS(G)
- G^T = transpune(G)
- DFS(G^T) (în bucla principală se tratează nodurile în ordinea descrescătoare a timpilor de finalizare de la primul DFS)

Componentele conexe sunt reprezentate de pădurea de arbori generați de DFS(G^T).

Algoritm:
https://www.geeksforgeeks.org/strongly-connected-components/

COMPLEXITATE: O(n+m)

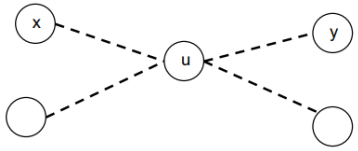
n = număr de noduri

Cheatsheet PA

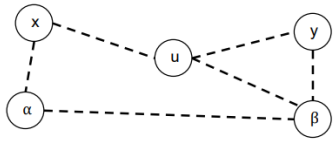
m = număr de muchii

PUNCT DE ARTICULAȚIE

Definiție: G = (V,E) graf neorientat, u∈V. U este punct de articulație dacă ∃ x,y∈V, x ≠ y, x ≠ u, y ≠ u, a.î. ∀ x..y în G trece prin u.



Orice drum x..y trece prin u → u este punct de articulație.

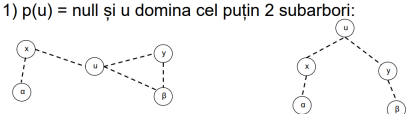


Exista x..α..y care nu trece prin u → u nu mai este punct de articulație!

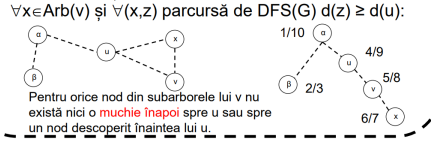
Teorema 5.15: G = (V,E), graf neorientat și u∈V. U este punct de articulație în G ⇔ în urma DFS în G una din proprietățile de mai jos este satisfăcută:

- p(u) = null și u domină cel puțin 2 subarbori;
- p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) avem d(z) ≥ d(u).

1) p(u) = null și u domina cel puțin 2 subarbori:



2) p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) d(z) ≥ d(u):



Pentru orice nod din subarborele lui v nu există nici o muchie înapoi spre u sau spre un nod descoperit înaintea lui u.

Structuri de date

Low(u) = min{d(v) | v descoperit pornind din u în cursul DFS și c(v) ≠ alb}

Subarb(u) = numărul subarborilor dominați de u (dacă este ≥ 2, atunci avem un punct de articulație).

Algoritmul lui Tarjan

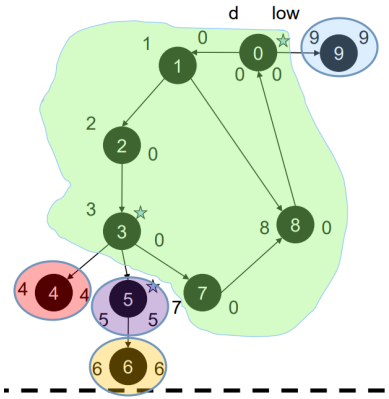
```
index = 0  // nivelul pe care este nodul în arborele DFS
S = empty // se folosește o stivă care se inițializează cu Ø
Pentru fiecare v din V
    Dacă (v.index e nedefinit) atunci // se pornește DFS din fiecare nod pe care
        Tarjan(v)                     // nu l-am vizitat încă

Tarjan(v)
    v.index = index // se setează nivelul nodului v
    v.lowlink = index // reține strămoșul nodului v
    index = index + 1  // incrementez nivelul
    S.push(v) // introduc v în stivă
    Pentru fiecare (v, v') din E // se prelucrează succesorii lui v
        Dacă (v'.index e nedefinit sau v' e în S) atunci // CTC deja identificate sunt ignorate
            Dacă (v'.index e nedefinit) atunci Tarjan(v') // dacă nu a fost vizitat v' intru în  recursivitate
            v.lowlink = min(v.lowlink, v'.lowlink) //actualizez strămoșul
    Dacă (v.lowlink == v.index) atunci // printez CTC începând de la coadă spre rădăcină
        print "CTC:"
        Repetă
            v' = S.pop // extrag nodul din stiva și îl printez
            print v'
        Până când (v' == v) // până când extrag rădăcina
```

low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}

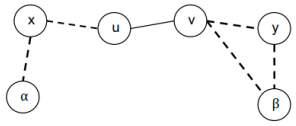
https://www.geeksforgeeks.org/tarjan-algorithm-find-strongly-connected-components/

În urma aplicării algoritmului lui Tarjan, se obține:



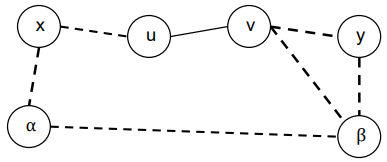
PUNȚI

Definiție: G = (V,E), graf neorientat și (u,v) ∈ E. (u,v) este punte în G ⇔ ∃ x,y ∈ V, x ≠ y, a.î. ∀ x..y conține muchia (u,v).



Orice drum x..y trece prin (u,v) =>(u,v) este punte

Cheatsheet PA



(u,v) nu este punte

Algoritm

Punți(G)

- V = noduri(G) // inițializări
- Timp = 0;
- **Pentru fiecare** nod u (u ∈ V)
  - c(u) = alb;
  - d(u) = 0;
  - p(u) = null;
  - low(u) = 0;
  - punte(u) =0; // înlocuiește: subarb(u) = 0; art(u) = 0;
- **Pentru fiecare** nod u (u ∈ V)
  - **Dacă** c(u) e alb
    - Explorează(u)

Explorează(u)

- d(u) = low(u) = timp++; // inițializări
- c(u) = gri;
- **Pentru fiecare** nod v (v ∈ succs(u))
  - **Dacă** c(v) e alb
    - p(v) = u; // se elimină: subarb(u)++;
    - Explorează(v);
    - low(u) = min{low(u), low(v)} // actualizare low
    - **Dacă** (low(v) > d(u)) punte(v) = 1;

    // în loc de: **Dacă**(p(u) != null && low(v) ≥ d(u))
  - **Altfel**
    - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

DRUMURI DE COST MINIM

TIPURI

Drumuri punct – multipunct: pentru un nod dat s ∈ V, să se găsească un drum de cost minim de la s la ∀u ∈ V; Dijkstra, Bellman-Ford

Drumuri multipunct – punct: pentru un nod dat e ∈ V, să se găsească un drum de cost minim de la ∀u ∈ V la e; G^T și apoi 1

Drumuri punct – punct: pentru două noduri date u și v ∈ V, să se găsească un drum u..v de cost minim; Folosind 1

Drumuri multipunct – multipunct: ∀u, v ∈ V, să se găsească un drum u..v de cost minim. Floyd-Warshall

Drumuri de cost maxim!

1) **Punct - multipunct** -> Dijkstra, Bellman-Ford
2) **Multipunct - punct** -> transpunem graful și aplicăm unul din cei 2 algoritmi de la 1
3) **Punct - punct** -> folosim 1
4) **Multipunct - multipunct** -> Floyd-Warshall
5) **Cost maxim** ->

ALGORITMUL LUI DIJKSTRA

Se folosește **NUMAI** pentru costuri pozitive (w(u,v) > 0, ∀u,v∈V).

Dijkstra(G,s)

- **Pentru fiecare** (u ∈ V) // inițializări
  - d[u] = ∞; p[u] = null;
- d[s] = 0;
- Q = construiește_coada(V) // coadă cu priorități
- **Cât timp** (Q != ∅)
  - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
  - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
  - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
    - **Dacă** (d[v] > d[u] + w(u,v))
      - d[v] = d[u] + w(u,v) // actualizez distanța
      - p[v] = u // și părintele

https://infoarena.ro/job_detail/2376287?action=view-source

Complexitate?
Vectori - $O(V^2)$
HB - O(E logV)
HF - O(V log V+E)

Algoritmul lui Bellman-Ford

https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/

Cheatsheet PA

BellmanFord(G,s) // G=(V,E),s=sursa
- **Pentru fiecare** nod v (v ∈ V) // inițializări
  - d[v] = ∞;
  - p[v] = null;
- d[s] = 0; // actualizare distanță de la s la s
- **Pentru** *i* **de la** *1* **la** |V| *-1* // pentru fiecare pas pornind din s // spre restul nodurilor se încearcă construcția unor drumuri // optime de dimensiune i
  - **Pentru fiecare** (u,v) din E // pentru arcele ce pleacă de la nodurile deja considerate
    - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele corespunzătoare
      - d[v] = d[u] + w(u,v);
      - p[v] = u;
- **Pentru fiecare** (u,v) din E
  - **Dacă** d[v] > d[u] + w(u,v) **atunci**
    - **Eroare** ("ciclu negativ");

Teorema. G = (V,E), w : E → ℜ funcție de cost asociată. Algoritmul Bellman-Ford aplicat acestui graf plecând din sursa s nu returnează EROARE dacă G nu conține cicluri negative, iar la terminare d[v] = δ(s,v) pentru ∀ v ∈ V. Dacă G conține cel puțin un ciclu negativ accesibil din s, atunci algoritmul întoarce EROARE.

BellmanFordOpt2(G,s)
- **Pentru fiecare nod v** (v ∈ V)
  - d[v] = ∞;
  - p[v] = null;
  - marcat[v] = false; // marcăm nodurile pentru care am făcut relaxare
- Q = ∅; // coadă cu priorități
- d[s] = 0; marcat[s] = true; Introdu(Q,s);
- **Cât timp** (Q != ∅)
  - u = ExtrageMin(Q); marcat[u] = false; // extrag minimul
  - **Pentru fiecare nod v** (v ∈ succesorilor lui u)
    - **Dacă** d[v] > d[u] + w(u,v) **atunci** // relaxez arcele ce pleacă din
      - d[v] = d[u] + w(u,v);
      - p[v] = u;
      - **Dacă** (marcat[v] == false) {marcat[v] = true; Introdu(Q,v);}

Observație: nu mai detectează cicluri negative!

FLOYD-WARSHALL (ROY FLOYD)

Calculează distanțele minime între oricare 2 noduri dintr-un graf (drumuri optime multipunct-multipunct).

Idee: la pasul k se calculează cel mai bun cost între u și v folosind cel mai bun cost u..k și cel mai bun cost k..v calculat până în momentul respectiv.

Rezultatele furnizate sunt corecte dacă algoritmul se aplică pe grafuri ce nu conțin cicluri de cost negativ.

Notații

G = (V,E); V = {1, 2, …, n};

w : V x V → ℜ; w(i, i) = 0; w(i, j) = ∞ dacă (i,j) ∉ E;

$d^k(i,j)$ = costul drumului i..j construit astfel încât drumul trece doar prin noduri din mulțimea {1, 2, .., k};

δ(i,j) = costul drumului optim i..j; δ(i,j) = ∞ dacă ∄ i..j;

$\delta^k(i,j)$ = costul drumului optim i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}; $\delta^k(i,j)$ = ∞ dacă ∄ un astfel de drum i..j;

$p^k(i,j)$ = predecesorul lui j pe drumul i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}.

Recurență

- $d^0(i,j) = w(i,j)$;
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$,

Soluție finală: ~~$d^n(i,j) = \delta(i,j)$~~;

Algoritm

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - $d^0(i,j) = w(i,j)$
    - **Dacă** (w(i,j) == ∞)
      - $p^0(i,j)$ = null;
    - **Altfel** $p^0(i,j) = i$;

Pentru k **de la** 1 **la** n
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - **Dacă** ($d^{k-1}(i,j) > d^{k-1}(i,k) + d^{k-1}(k,j)$)
      - $d^k(i,j) = d^{k-1}(i,k) + d^{k-1}(k,j)$
      - $p^k(i,j) = p^{k-1}(k,j)$; // și actualiz
    - **Altfel**
      - $d^k(i,j) = d^{k-1}(i,j)$
      - $p^k(i,j) = p^{k-1}(i,j)$;

Complexitate -> O(V^3)

Se poate reduce complexitatea spațială, folosind o singură matrice de dimensiune n x n => complexitate spațială O(N^2)

Problemă: în pasul k, pentru k < i și k < j, d(i,k) și d(k,j) folosite la calculul d(i,j) sunt $d^k(k,j)$ și $d^k(i,k)$ în loc de $d^{k-1}(k,j)$ și $d^{k-1}(i,k)$. Dacă dem. că $d^k(k,j) = d^{k-1}(k,j)$ și $d^k(i,k) = d^{k-1}(i,k)$, atunci putem folosi o singură matrice.

Dar: $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
- $d^k(k,j) = \min\{d^{k-1}(k,j), d^{k-1}(k,k) + d^{k-1}(k,j)\} = d^{k-1}(k,j)$
- $d^k(i,k) = \min\{d^{k-1}(i,k), d^{k-1}(i,k) + d^{k-1}(k,k)\} = d^{k-1}(i,k)$

Alg. după optimizare

```
Pentru i de la 1 la n
    Pentru j de la 1 la n // inițializări
        d(i,j) = w(i,j)
        Dacă (w(i,j) == ∞)
            p(i,j) = null;
        Altfel p(i,j) = i;
Pentru k de la 1 la n
    Pentru i de la 1 la n
        Pentru j de la 1 la n
            Dacă (d(i,j) > d(i,k) + d(k,j)) // determ
                d(i,j) = d(i,k) + d(k,j)
                p(i,j) =  p(k,j); // și actualizăm pă
```

Închidere tranzitivă

Fie G = (V,E). Închiderea tranzitivă a lui E e un G* = (V,E*), unde

$$E^*(i,j)=\begin{cases}1, \text{ dacă } \exists\ i..j\\ 0, \text{ dacă } \nexists\ i..j\end{cases}$$

Obținerea închiderii tranzitive a unui graf se face prin modificarea algoritmului Floyd-Warshall, astfel:

min $\Longrightarrow$ operatorul boolean sau ($\vee$)

\+ $\Longrightarrow$ operatorul boolean și ($\wedge$)

```
Închidere_tranzitivă(G)
    Pentru i de la 1 la n
        Pentru j de la 1 la n
            E* (i,j) = (((i,j) ∈ E) ∨ (i = j)) // inițializări
    Pentru k de la 1 la n
        Pentru i de la 1 la n
            Pentru j de la 1 la n
                E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))
```

Complexitate? $O(V^3)$ Complexitate spațială? $O(V^2)$

Algoritmul lui Johnson

Este folosit pentru grafuri rare. Mai eficient decât Floyd-Warshall pentru acest tip de grafuri.

Complexitate: $O(V^2 \log V + VE)$

Dacă graful conține doar arce de cost pozitiv - se aplică Dijkstra pentru fiecare nod

Astfel, se recalculează costurile muchiilor astfel încât să se respecte următoarele proprietăți:

$w_1(u,v) \geq 0,\ \forall\ (u,v) \in E;$

p este drum minim utilizând w $\Leftrightarrow$ p este drum minim utilizând w_1.

Adăugăm un nod nou în graf și legăm muchii de la el la toate nodurile din graf cu costul 0. Aplicăm algoritmul lui Bellman-Ford și verificăm dacă graful conține cicluri negative. Dacă nu conține determinăm în urma alg. BF vectorul h(x) = distanța minimă de la nodul adăugat la nodul x. Calculăm noua funcție de costuri astfel:

$$w_1(u,v) = w(u,v) + h(u) - h(v);$$

Se aplică alg Johnson pentru graf cu arce de cost pozitiv (după ce eliminăm nodul adăugat inițial).

```
Johnson(G)
    // Construim G' = (V',E');
    V' = V ∪ {s}; // adăugăm nodul s
    E' = E ∪ (s,u), ∀u∈V; w(s,u) = 0; // și îl legăm de toate nodurile
    Dacă BF(G',s) e fals // aplic BF pe G'
        Eroare "ciclu negativ"
    Altfel
        Pentru fiecare v ∈ V
            h(v) = δ(s,v); // calculat prin BF
        Pentru fiecare (u,v) ∈ E
            w1(u,v) = w(u,v) + h(u) - h(v) // calculez noile costuri pozitive
        Pentru fiecare (u ∈ V)
            Dijkstra(G,w1,u) // aplic Dijkstra pentru fiecare nod
            Pentru fiecare (v ∈ V)
                d(u,v) = δ1(u,v) + h(v) - h(u) // calculez costurile pe graful inițial
```

Arbore minim de acoperire

Cheatsheet PA

Definiție: Un arbore liber se numește arbore de acoperire dacă V' = V.

Definiție: Un arbore de acoperire (Arb) se numește arbore minim de acoperire (notăm AMA) dacă Arb ∈ ARB(G) a.î. C(Arb) = min{C(Arb') | Arb' ∈ ARB(G)}.

G = (V,E), C = (V',E') – ciclu în G; e ∈ E' a.î. w(e) = max {w(e') | e' ∈ E'} => e ∉ Arb(G) unde Arb(G) = AMA în G.

Proprietăți (I)

G = (V,E), C = (V',E') – ciclu în G; e ∈ E' a.î. w(e) = max {w(e') | e' ∈ E'} => e ∉ Arb(G) unde Arb(G) = AMA în G.

Proprietăți (II)

G = (V,E), S = (V',E') un AMA parțial al lui G, V' ⊂ V; e = (u,v) a.î. e ∉ E' și (u ∈ V' și v ∉ V') sau (u ∉ V' și v ∈ V') cu proprietatea că: w(u,v) = min{w(u',v')| (u'∈V' și v'∉V') sau (u'∉V' și v'∈V')} => (u,v) ∈ AMA.

Algoritmi AMA / APM

2 algoritmi greedy

Prim: se pornește cu un nod și se extinde pe rând cu muchiile cele mai ieftine care au un singur capăt în mulțimea de muchii deja formată (Proprietatea 2). Algoritmul este asemănător algoritmului Dijkstra.

Kruskal: inițial toate nodurile formează câte o mulțime și la fiecare pas se reunesc 2 mulțimi printr-o muchie. Muchiile sunt considerate în ordinea costurilor și sunt adăugate în arbore doar dacă nu creează ciclu (Proprietatea 1).

PRIM

- asemănător algoritm Dijkstra

Prim(G,w,s) Implementare în Java la [

- A = ∅ // inițializare AMA
- **Pentru fiecare** (u ∈ V)
  - d[u] = ∞; p[u] = null // inițializăm distanța și părintele
- d[s] = 0; // nodul de start are distanța 0
- Q = constrQ(V, d); // ordonată după costul muchiei // care unește nodul de AMA deja creat
- **Cât timp** (Q != ∅) // cât timp mai sunt noduri neadăugate
  - u = ExtrageMin(Q); // extrag nodul aflat cel mai aproape
  - A = A ∪ {(u,p[u])}; // adaug muchia în AMA
  - **Pentru fiecare** (v ∈ succs(u))
    - **Dacă** d[v] > w(u,v) **atunci**
      - d[v] = w(u,v); //+ d[u] // actualizăm distanțele și părinții nodurilor
      - p[v] = u; // adiacente care nu sunt în AMA încă
- **Întoarce** A - {(s,p(s))} // prima muchie adăugată

https://www.geeksforgeeks.org/prims-minimum-spanning-tree-mst-greedy-algo-5/

Complexitate Prim

- Depinde de implementare (vezi Dijkstra)
  - Matrice de adiacență $O(V^2)$
  - Heap binar $O(ElogV)$
  - Heap Fibonacci $O(VlogV+E)$
- Concluzii
  - Grafuri dese
    - Matrice de adiacență preferată
  - Grafuri rare
    - Heap binar sau Fibonacci

KRUSKAL

Kruskal(G,w) Implementare în Java la [4] !

- A = ∅; // inițializare AMA
- **Pentru fiecare** (v ∈ V)
  - Constr_Arb(v) // creează o mulțime formată din nodul respectiv // (un arbore cu un singur nod)
- Sortează_asc(E,w) // se sortează muchiile în funcție de // costul lor
- **Pentru fiecare** ((u,v) ∈ E) // muchiile se extrag în ordinea // costului
  - **Dacă** Arb(u) != Arb(v) **atunci** // verificăm dacă se creează ciclu
    - Arb(u) = Arb(u) ∪ Arb(v) // se reunesc mulțimile de noduri (arborii)
    - A = A ∪ {(u,v)} // se adaugă muchia sigură în AMA
- **Întoarce** A

Elementele algoritmului:

- Sortarea muchiilor: O(ElogE) ≈ O(ElogV)
- Arb(u) = Arb(v) – compararea a 2 mulțimi disjuncte {1,2,3} {4,5,6} – mai precis trebuie identificat dacă 2 elemente sunt în aceeași mulțime
- Arb(u) ∪ Arb(v) – reuniunea a 2 mulțimi disjuncte într-una singură

Alternative pentru operațiile cu mulțimi

id[i] = părintele lui i

pentru rădăcină id[i] = i

Arb(i) // identificarea rădăcinii unei componente
- **Cât timp** (i != id[i]) i = id[i];
- **Întoarce** i

Comparare (u, v)
- **Întoarce** Arb(u) != Arb(v)

Reuniune (u,v) // implică identificarea rădăcinii
- v = Arb(v)
- id[v] = u;

Worst case comparare + reuniune - O(V)

Optimizare

Se menține numărul de noduri din fiecare subarbore. Se adaugă arborele mic la cel mare pentru a face mai puține căutări → înălțimea arborelui e mai mică și numărul de căutări scade de la V la lg V. Complexitatea pentru comparare și reuniune devine O(lgV).

Identificarea rădăcinii:
- Arb(i)
  - **Cât timp** (i != id[i])
    - id[i] = id[id[i]];
    - i = id[i];
  - **Întoarce** i

Complexitate Kruskal - O(ElogV) - complexitatea este dată de sortarea muchiilor

Aplicație Kruskal

K-clustering
- Împărțirea unui set de obiecte în k grupuri astfel încât obiectele din cadrul unui grup să fie "apropiate" considerând o "distanță" dată.

Se formează V clustere (un cluster per obiect). Găsește cele mai apropiate 2 obiecte din clustere diferite și unește cele 2 clustere. Se oprește când au mai rămas k clustere. → chiar algoritmul Kruskal

FLUX

G(V,E) graf orientat;

c(u,v) ≥ 0 ∀(u,v) – c = capacitatea arcelor;

Dacă (u,v) ∉ E → c(u,v) = 0;

S – sursa traficului;

T – destinația traficului (drena);

Presupunem că ∀u ∈ V \ {s, t} ∃ s..u..t.

f: V x V→ℜ - fluxul prin rețeaua G;

Proprietăți:
- ∀u, v ∈ V, f(u,v) ≤ c(u,v) (fluxul printr-un arc este mai mic sau egal cu capacitatea arcului) – respectarea capacității arcelor;
- ∀u, v ∈ V, f(u,v) = -f(v,u) – simetria fluxului;
- Σf(u,v) = 0 pentru ∀u ∈ V \ {s,t} – conservarea fluxului.

f(u,v) – fluxul din u spre v;

$f_i(u) = \Sigma f(v,u)$ – fluxul total care intra în nodul u;

$f_o(u) = \Sigma f(u,v)$ – fluxul total care iese din nodul u;

Valoarea totală a fluxului:
- $|f| = \Sigma f(s,v) = f_o(s)$;
- |f| = fluxul ce părăsește sursa;
- Cf. proprietăților P1-P3: $|f| = \Sigma f(s,v) = \Sigma f(v,t) = f_i(t)$.

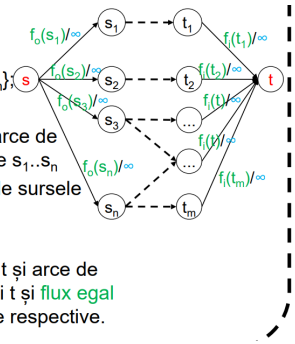
Pentru surse multiple / destinații multiple se adaugă o sursă / destinație unică.

Surse multiple $\{s_1, s_2, \ldots, s_n\}$;

Destinații multiple $\{t_1, t_2, \ldots, t_m\}$;

Se adaugă o sursă unică cu arce de capacitate infinită spre sursele $s_1..s_n$ și flux egal cu fluxul generat de sursele respective;

Se adaugă o destinație unică t și arce de capacitate infinită între $t_1..t_m$ și t și flux egal cu fluxul ce intră în destinațiile respective.



Operații cu fluxuri

Cheatsheet PA

X,Y – mulțimi de noduri;

$f(X,Y) = \Sigma_{x\in X}\Sigma_{y\in Y} f(x,y)$ = fluxul între X și Y;

Operații:

- $\forall X \in V$: $f(X,X) = 0$;
- $\forall X,Y \in V$: $f(X,Y) = -f(Y,X)$;
- $\forall X,Y,Z \in V$ și $Y \subseteq X$:
  - $f(X \setminus Y, Z) = f(X,Z) - f(Y,Z)$;
  - $f(Z, X \setminus Y) = f(Z,X) - f(Z,Y)$;
- $\forall X,Y,Z \in V$ și $X \cap Y = \emptyset$:
  - $f(X \cup Y, Z) = f(X,Z) + f(Y,Z)$;
  - $f(Z, X \cup Y) = f(Z,X) + f(Z,Y)$
- $f(s,V) = f(V,t)$

Arc rezidual -> fluxul pe acest arc se poate mări

Definiție: Un arc (u,v) pentru care f(u,v) < c(u,v) se numește arc rezidual.

Definiție: Cantitatea cu care se poate mări fluxul pe arcul (u,v) se numește capacitatea reziduală a arcului (u,v) ($c_f(u,v)$):

$$c_f(u,v) = c(u,v) - f(u,v)$$

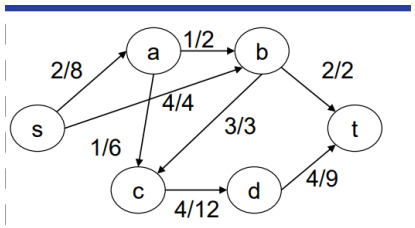
Definiție: Rețeaua reziduală ($G_f = (V,E_f)$) este o rețea de flux formată din arcele ce admit creșterea fluxului:

$E_f = \{(u,v) \in V \times V \mid c_f(u,v) > 0\}$.

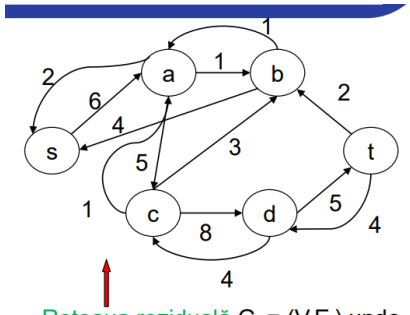
Definiție: Cale reziduală e un drum s..t $\subseteq G_f$, unde $c_f(u,v)$ este capacitatea reziduală a arcului (u,v).

Definiție: Capacitatea reziduală a căii = capacitatea reziduală minimă de pe calea s..t descoperită.

Exemplu rețea reziduală



$c_f(u,v) = c(u,v) - f(u,v)$



Rețeaua reziduală $G_f = (V,E_f)$ unde
$E_f = \{(u,v) \in V \times V \mid c_f(u,v) > 0\}$

Calea reziduală: s → a → c → d → t
Capacitatea reziduală a căii:
$c_f(p) = \min\{6, 5, 8, 5\} = 5$

Lemă 5.17: G – rețea de flux, f flux în G, p = s..t – cale reziduală în G_f, $f_p: V \times V \to \Re$ se definește ca fiind:

$$f_p(u,v) = \begin{cases} c_f(p), \text{ dacă } (u,v) \in p \\ -c_f(p), \text{ dacă } (v,u) \in p \\ 0, \text{ dacă } (u,v) \text{ și } (v,u) \notin p \end{cases}$$

f_p = flux în G_f; $|f_p| = c_f(p)$

Corolar 5.4: $f' = f + f_p$ = flux în G, astfel încât
$|f'| = |f| + |f_p| > |f|$

Această Lemă ne spune cum se definește fluxul printr-o rețea reziduală.

Calcularea fluxului maxim

Metoda Ford-Fulkerson

- f(u,v) = 0 ∀ (u,v) // inițializarea fluxului
- **Repetă** // creștere iterativă a fluxului
  - găsește un drum s..p..t pe care se poate mări fluxul (cale reziduală)
  - f = f + flux(s..p..t)
- **Până când** nu se mai poate găsi nici un drum s..p..t
- **Întoarce** f

FORD-FULKERSON

Cheatsheet PA

Ford – Fulkerson(G,s,t)

- **Pentru fiecare** (u,v) din E
  - f(u,v) = f(v,u) = 0 // inițializare
- **Cât timp**
  - Există o cale reziduală p între s..t în G_f
    - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // capacitatea reziduală
    - **Pentru fiecare** (u,v) din p
      - f(u,v) = f(u,v) + $c_f(p)$
      - f(v,u) = -f(u,v)

Complexitate?

- **Întoarce** |f|

Complexitate $O(E * f_{max})$

f_{max} = fluxul maxim

Îmbunătățiri:

- Se aleg căile reziduale cu capacitate maximă – complexitatea va depinde în continuare de f_{max} și de valoarea capacităților;
- Se aleg căile reziduale cele mai scurte → în acest caz complexitatea nu mai depinde de f_{max} ci numai de numărul de arce (ex. Edmonds-Karp: identificarea căilor reziduale minime prin aplicarea unui BFS)

Edmonds – Karp(G, s, t)

- **Pentru fiecare** (u,v) din E
  - f(u,v) = f(v,u) = 0 // inițializare
- **Cât timp**
  - Există căi reziduale între s..t în G_f
    - Determină calea reziduală de lungime minimă p aplicând BFS
    - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // capacitatea reziduală
    - **Pentru fiecare** (u,v) din p
      - f(u,v) = f(u,v) + $c_f(p)$
      - f(v,u) = -f(u,v)

Complexitate?

- **Întoarce** |f|

https://infoarena.ro/job_detail/2391089?action=view-source

Preflux

Definiție: Preflux = f: V x V → $\Re$ astfel încât să fie satisfăcute restricțiile:

- $f(u,v) \leq c(u,v)$, $\forall (u,v) \in E$ – respectarea capacității arcelor;
- $f(u,v) = -f(v,u)$, $\forall u,v \in V$ – simetria fluxului;
- $\Sigma_{v \in V} f(v,u) \geq 0$, $\forall u \in V \setminus \{s\}$ – conservarea fluxului.

Definiție: O funcție h: V → N este o funcție de înălțime dacă îndeplinește restricțiile:

- h(s) = |V| – fixă;
- h(t) = 0 – fixă;
- $h(u) \leq h(v) + 1$ pentru orice arc rezidual $(u,v) \in G_f$ – variabilă.

Lema 5.19: G – rețea de flux, h: V → N este o funcție de înălțime. Dacă $\forall u, v \in V$, h(u) > h(v) + 1 atunci arcul (u,v) nu este arc rezidual.

Pompare(u,v) // pompează fluxul în exces (e(u) > 0)
// are loc doar dacă diferența de înălțime dintre u și v este 1
// (h(u) = h(v) + 1), altfel nu e arc rezidual și nu ne interesează

- d = min(e(u), $c_f(u,v)$); // cantitatea de flux pompată
- f(u,v) = f(u,v) + d; // actualizare flux pe arcul (u,v)
- f(v,u) = -f(u,v); // respectarea simetriei
- e(u) = e(u) – d; // actualizare supraîncărcare la sursă
- e(v) = e(v) + d; // actualizare supraîncărcare la destinație

Înălțare(u) // mărește h(u) dacă u are flux în exces
// (e(u) > 0) și $u \notin \{s, t\}$ $\forall (u,v) \in G_f$ avem $h(u) \leq h(v)$

- $h(u) = 1 + \min\{h(v) \mid (u,v) \in G_f\}$

Init_preflux(G, s, t)

- **Pentru fiecare** (u ∈ V)
  - e(u) = 0 // inițializare exces flux în nodul u
  - h(u) = 0 // inițializare înălțime nod u
- **Pentru fiecare** ((u,v) ∈ E) // inițializare fluxuri
  - f(u,v) = 0
  - f(v,u) = 0
- h(s) = |V| // inițializare înălțime sursă
- **Pentru fiecare** (u ∈ succs(s) \ {s})
  - f(s,u) = c(s,u); // actualizare flux
  - f(u,s) = -c(s,u);
  - e(u) = c(s,u) // actualizare exces

Complexitat

Pompare_preflux(G, s, t)

- Init_preflux(G, s, t) // inițializarea prefluxului
- **Cât timp** (1) // cât timp pot face pompări sau înălțări
  - **Dacă** ($\exists u \in V \setminus \{s, t\}$, $v \in V$ | e(u) > 0 **și** $c_f(u,v) > 0$ **și** h(u) = h(v) + 1) // încerc să pompez
    - Pompare(u,v); **continuă;**
  - **Dacă** ($\exists u \in V \setminus \{s, t\}$, $v \in V$ | e(u) > 0 **și** $\forall (u,v) \in E_f$, $h(u) \leq h(v)$)
    - Înălțare(u); **continuă;** // încerc să înalț
  - **Întrerupe;** // nu mai pot face nimic → am ajuns la flux max
- **Întoarce** e(t) // e(t) = |f| = fluxul total în rețea

Init_preflux: O(E)
Pompare(u,v): O(1)
Înălțare(u): O(V) – implică găsirea minimului dintre nodurile succesoare
Cât timp: [vezi Cormen]

- Câte înălțări?
  - Care e înălțimea maximă? 2 (|V| - 1) – drum rezidual de lungime maximă
  - Care este numărul maxim total de înălțări? (2 |V| - 1) (|V| - 2) ~ 2 $|V|^2$
- Câte pompări?
  - Pompări saturate: 2 |V| |E| - de câte ori un arc poate fi saturat? (în funcție de suma h(u) + h(v))
  - Pompări nesaturate: 4 $|V|^2$ (|V| + |E|) – sumă înălțimi noduri excedentare

Complexitate totală: $O(V^2 * E)$ [vezi Cormen]

ALGORITMI EURISTICI

Cheatsheet PA

Stare a problemei = abstractizare a unei configurații valide a universului problemei, configurație ce determină univoc comportarea locală a fenomenului descris de problemă.

Spațiul stărilor = graf în care nodurile corespund stărilor problemei, iar arcele desemnează tranzițiile valide între stări.

Descriere
- Nodul de start (starea inițială);
- Funcție de expandare a nodurilor (produce lista nodurilor asociate stărilor valide în care se poate ajunge din starea curentă);
- Predicat de testare dacă un nod corespunde unei stări soluție.

Definiție: Dacă explorarea se bazează pe informația acumulată în cursul explorării, informație prelucrată euristic (costuri)→ algoritm informat.

Definiție: Dacă explorarea este 'la întâmplare' → algoritm neinformat.

Definiție: Dacă algoritmul de explorare are posibilitatea să abandoneze calea curentă de rezolvare și să revină la o cale anterioară → algoritmi tentativi.

Definiție: Altfel (algoritmul avansează pe o singură direcție) → algoritmi irevocabili.

OPEN = mulțimea (lista) nodurilor explorate (frontiera dintre zona cunoscută și cea necunoscută).

CLOSED = mulțimea (lista) nodurilor expandate (regiunea cunoscută în totalitate).

Explorarea zonelor necunoscute se face prin alegerea și expandarea unui nod din OPEN. După expandare, nodul respectiv e trecut în CLOSED.

Majoritatea algoritmilor tentativi folosesc lista OPEN, dar doar o parte folosesc lista CLOSED.

Definiție: Algoritm complet = algoritm de explorare care garantează descoperirea unei soluții, dacă problema acceptă soluție.
- Algoritmii irevocabili sunt mai rapizi și consumă mai puține resurse decât cei tentativi, dar nu sunt completi pentru că pierd informație.

Definiție: Algoritm optimal = algoritm de explorare care descoperă soluția optimă a problemei.

Explorare(StInit, test_sol)
- OPEN = {constr_nod(StInit)}; // starea inițiala
- **Cât timp** (OPEN ≠ Ø) // mai am noduri de prelucrat
  - nod = selecție_nod(OPEN); // aleg un nod
  - **Dacă** (test_sol(nod)) **Întoarce** nod; // am găsit o soluție
  - OPEN = OPEN \ {nod} U expandare{nod}; // extind căutarea
- **Întoarce** insucces; // nu s-a găsit nicio soluție

Explorarea tentativă completă BF* (BEST FIRST)

BF*(StInit, f, test_sol)
- nod = constr_nod(StInit); // starea inițială
- π(nod) = null;
- OPEN = {nod}; // noduri explorate dar neexpandate
- CLOSED = Ø; // noduri expandate — Inițializări
- **Cât timp** (OPEN ≠ Ø)
  - nod = selecție_nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
  - **Dacă** (test_sol(nod)) **întoarce** nod; — Soluția
  - OPEN = OPEN \ {nod};
  - CLOSED = CLOSED U {nod};
  - succs = expand(nod); — Continuarea căutării
  - **Pentru fiecare** (succ ∈ succs)
    - **Dacă** (succ ∉ CLOSED U OPEN) **atunci** — Nod nou
      - OPEN = OPEN U {succ}; π(succ) = nod;
    - **Altfel**
      - succ' = apariția lui succ în CLOSED U OPEN
      - **Dacă** (f(succ) < f(succ')) // am găsit o cale mai bună către succ și // redeschidem nodul
        - π(succ') = nod; // actualizez părintele
        - f(succ') = f(succ); // și costul nodului — Actualizări
        - **Dacă** (succ' ∈ CLOSED) // dacă era considerat expandat, îl redeschid
          - CLOSED = CLOSED \ {succ'}; OPEN = OPEN U {succ'}; — Reprelucrare
  - **Întoarce** insucces; — Insucces

ex: Best-first cu diverse euristici

Optimalitate?
Completitudine?
Complexitate?

Algoritmul este complet dar nu este optim → optimalitatea depinde de euristica f!

Complexitate: $O(b^{d+1})$

A*

Cheatsheet PA

A*(StInit, h, test_sol)

- n_0 = constr_nod(StInit); // starea inițială

Inițializări

- $f(n_0) = h(n_0)$; $g(n_0) = 0$; $\pi(n_0)$ = null; // euristici
- OPEN = $\{n_0\}$; CLOSED = Ø; // și cozi

- **Cât timp** (OPEN ≠ Ø) // mai am noduri de prelucrat
  - nod = selecție_nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
  - **Dacă** (test_sol(nod)) **întoarce** nod;

Soluția

  - OPEN = OPEN \ {nod}; // updatez OPEN
  - CLOSED = CLOSED U {nod}; // și CLOSE
  - succs = expand(nod); // determin nodurile succesoare

Continuarea căutării

  - **Pentru fiecare** (succ ∈ succs) { // prelucrare succs
    - g_succ = g(nod) + c(nod, succ); // calculez g
    - f_succ = g_succ + h(succ); // calculez f = g + h

Prelucrare succesori

    - **Dacă** (succ ∉ CLOSED U OPEN) **atunci** // nod nou descoperit →
      - OPEN = OPEN U {succ}; // îl bag în OPEN
        g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod;

Nod nou

    - **altfel** // a mai fost prelucrat
      - **Dacă** (g_succ < g(succ)) // verific dacă noul g este mai mic decât // anteriorul
        g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod; // cale mai bună

Actualizări

Reprelucrare **Dacă** (succ ∈ CLOSED) // dacă era considerat expandat, îl redeschid
CLOSED = CLOSED \ {succ'}; OPEN = OPEN U {succ'};

- **Întoarce** Insucces;

Insucces

ex: A* cu diverse euristici